

Flash-KMeans: Fast and Memory-Efficient Exact K-Means

Shuo Yang*, Haocheng Xi*, Yilong Zhao, Muyang Li, Xiaoze Fan, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Kurt Keutzer, Song Han, Chenfeng Xu, Ion Stoica

*Equal Contribution

k -means has historically been positioned primarily as an offline processing primitive, typically used for dataset organization or embedding preprocessing rather than as a first-class component in online systems. In this work, we revisit this classical algorithm under the lens of modern AI system design and enable k -means as an online primitive. We point out that existing GPU implementations of k -means remain fundamentally bottlenecked by low-level system constraints rather than theoretical algorithmic complexity. Specifically, the assignment stage suffers from a severe IO bottleneck due to the massive explicit materialization of the $N \times K$ distance matrix in High Bandwidth Memory (HBM). Simultaneously, the centroid update stage is heavily penalized by hardware-level atomic write contention caused by irregular, scatter-style token aggregations. To bridge this performance gap, we propose flash-kmeans, an IO-aware and contention-free k -means implementation for modern GPU workloads. Flash-kmeans introduces two core kernel-level innovations: (1) *FlashAssign*, which fuses distance computation with an online argmin to completely bypass intermediate memory materialization; (2) *sort-inverse update*, which explicitly constructs an inverse mapping to transform high-contention atomic scatters into high-bandwidth, segment-level localized reductions. Furthermore, we integrate algorithm-system co-designs, including chunked stream overlap and cache-aware compile heuristic, to ensure practical deployability. Extensive evaluations on NVIDIA H200 GPUs demonstrate that flash-kmeans achieves up to $17.9\times$ end-to-end speedup over best baselines, while outperforming industry-standard libraries like cuML and FAISS by $33\times$ and over $200\times$, respectively. At the kernel level, FlashAssign and Sort-Inverse Update deliver up to $21.2\times$ and $6.3\times$ speedups. Beyond raw compute, our pipelined co-design enables seamless out-of-core execution on up to **one billion points** with a $10.5\times$ speedup, and our heuristics slash configuration tuning overhead by $175\times$ with near-zero performance degradation. By systematically restructuring execution around underlying hardware constraints, flash-kmeans delivers mathematically exact, scalable, and highly deployable acceleration across diverse AI workloads.

Correspondence: Shuo Yang at andy_yang@berkeley.edu, Chenfeng Xu at xuchenfeng@utexas.edu

Code: <https://github.com/svg-project/flash-kmeans>

1 Introduction

k -means (Lloyd, 1982; MacQueen, 1967) is one of the most classical and widely used clustering algorithms, and has long been treated as a mature, general-purpose component. In traditional settings, k -means typically appears as part of offline data processing pipelines (Sculley, 2010), and most optimizations focus on the algorithmic level, such as faster convergence, fewer distance evaluations, or better approximations (Elkan, 2003; Ding et al., 2015; Bachem et al., 2018). Although these algorithmic improvements successfully reduce theoretical FLOPs, many of them fail to translate into real end-to-end speedups on modern GPUs. This discrepancy arises because traditional optimizations are often incompatible with modern hardware design principles: dense compute (e.g., matrix multiplications via Tensor Cores) is exceedingly cheap (NVIDIA, 2021), while frequent, fragmented memory operations incur massive overheads. Consequently, lowering FLOPs does not necessarily correlate with wall-clock time reduction, and heavily overlooks the dominant constraints of memory bandwidth and data movement (Dao et al., 2022; Dao, 2023).

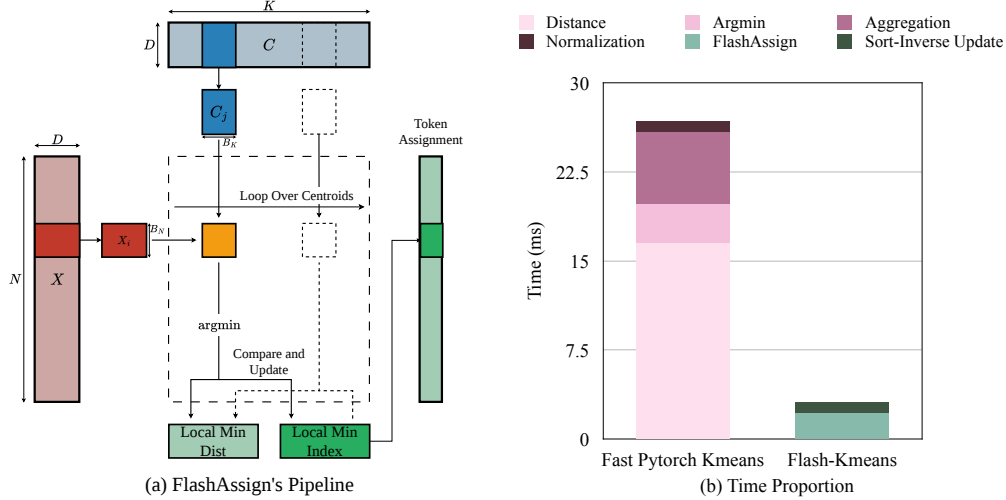


Figure 1 Overview of flash-kmeans and Performance Breakdown. (a) Inspired by IO-aware attention mechanisms, FlashAssign streams data blocks from HBM to SRAM, fusing distance computation with an online argmin operator to completely bypass the materialization of the massive $N \times K$ distance matrix. (b) Compared to standard k -means implementations, flash-kmeans drastically compresses both the assignment IO bottleneck and the update desynchronization bottleneck.

This performance gap caused by implementation-level factors is significantly amplified in modern AI workloads. k -means usage is undergoing a clear paradigm shift, evolving from an offline analytics tool into a high-frequency, online operator within training and inference pipelines (e.g., vector quantization and sparse routing) (Roy et al., 2020; Zhu et al., 2025; Liu et al., 2025). This shift manifests in three dimensions: (1) execution moves from CPUs to GPUs, creating clear potential to exploit hardware accelerators; (2) usage moves from offline to online invocation, making low latency per invocation strictly essential; and (3) computation moves to longer sequences and larger batches, making throughput and scalability crucial requirements.

To understand why standard implementations fail to meet these new requirements, we must first define the k -means workload. A standard Lloyd’s iteration consists of two distinct stages: an *assignment stage*, which computes the distance between every data point and all centroids to find the nearest cluster, and a *centroid update stage*, which is essentially an averaging operation that aggregates the features of all points assigned to each specific cluster to form the new centroids.

When profiling this standard pipeline on modern GPUs (Raschka et al., 2020; Johnson et al., 2019; Clavié and Warner, 2025), we find that the end-to-end performance is severely bottlenecked by kernel-level dataflow and write-path serialization. Concretely, this performance gap is driven by three primary challenges:

1. **IO-bound assignment in assignment stage:** Consider a workload consisting of N data points in d -dimensional space, clustered into K centroids. In each iteration, the assignment step compares every point against all K centroids, resulting in a computational cost of $O(NKd)$. Standard implementations execute this by first computing distances and then applying argmin, which explicitly materializes a massive distance matrix $D \in \mathbb{R}^{N \times K}$. Writing D to High Bandwidth Memory (HBM) and immediately reading it back incurs immense memory traffic. Under a representative setting with $N = 65536$, $K = 1024$, $d = 128$, $B = 32$, the distance computation itself takes only 2.6 milliseconds, whereas the memory time dominated by materializing and consuming D takes about 23 milliseconds.
2. **Atomic write contention in update stage:** The centroid update stage requires aggregating data grouped by cluster indices. Standard implementations typically perform scatter-style updates, where each thread atomically accumulates its point’s data into a shared sum and count buffer based on its cluster id. Consequently, many threads concurrently attempt to update the same centroid (especially for unbalanced, "hot" clusters), causing severe atomic contention and hardware-level serialization. On an NVIDIA H200 GPU, we measure only 50 GB/s effective bandwidth for this stage, drastically below the achievable

bandwidth of regular reductions.

3. **System-level constraints:** Beyond kernel-level bottlenecks, real-world deployments face severe system constraints. When the input batch cannot reside entirely in VRAM, chunk-wise execution introduces heavy host-to-device communication overheads. Furthermore, modern AI workloads often exhibit dynamic shapes, which amplify compilation and configuration tuning costs, drastically increasing the time-to-first-run and hurting practical usability.

To bridge this performance gap, we propose flash-kmeans, an efficient implementation of k -means designed for modern GPU workloads. Flash-kmeans does not alter the mathematical formulation of the standard Lloyd k -means, nor does it introduce approximations. Instead, it restructures the assignment stage, the centroid update stage, and the system execution path around the three hardware bottlenecks discussed above.

Concretely, we introduce three key techniques. First, to eliminate distance materialization, we propose *FlashAssign* (Figure 1a), which fuses streaming distance computation with an online argmin procedure. This avoids constructing the $N \times K$ intermediate matrix entirely, thereby eliminating the huge related overhead (Dao et al., 2022). Second, to resolve atomic contention, we introduce *sort-inverse update* (Figure 2b), which explicitly sorts the assignment vector by cluster id and performs aggregations in this sorted logical order. This rewrites high-contention, per-token atomic scatters into highly regular, segment-level localized merges. Finally, we introduce a set of *algorithm-system co-design* optimizations, including chunked stream overlap to hide PCIe communication overhead for large-scale execution, and a cache-aware compile heuristic to reliably match near-optimal configurations without expensive tuning, solving the dynamic deployment challenge.

We prototype flash-kmeans with customized GPU kernels and systematically evaluate it on an NVIDIA H200 GPU. We first conduct a kernel-level breakdown, revealing that FlashAssign speeds up the assignment kernel by up to $21.2\times$, and Sort-Inverse Update accelerates the centroid update kernel by up to $6.3\times$. Scaling to end-to-end iterations, we sweep across various numbers of points N , clusters K , feature dimensions d , and batch sizes B . flash-kmeans consistently achieves state-of-the-art performance across all representative workload regimes. As shown in Figure 1b, it outperforms the strongest baseline by up to $17.9\times$, and delivers massive speedups of up to $33\times$ and over $200\times$ against widely adopted industry-standard libraries like NVIDIA cuML (Raschka et al., 2020) and FAISS (Johnson et al., 2019), respectively. Beyond raw kernel speedups, our algorithm-system co-design optimizations prove critical for real-world deployment. In extreme large-scale out-of-core settings, our asynchronous data pipeline successfully scales to one billion points and delivers up to a $10.5\times$ end-to-end speedup by effectively overlapping PCIe communication with computation. Furthermore, under dynamic shape deployments, our cache-aware compile heuristic matches the optimally tuned kernel performance within a negligible 0.3% margin while slashing the compilation and configuration tuning overhead by up to $175\times$. Together, these results demonstrate that flash-kmeans delivers not only theoretical kernel efficiency, but also stable, highly deployable end-to-end acceleration for modern AI workloads.

2 Related Work

The evolution of K-Means workloads. Traditionally, k -means was deployed in offline data mining and classical computer vision (Lloyd, 1982; MacQueen, 1967). However, modern AI pipelines have transformed it into a high-frequency, online primitive across diverse domains. In large-scale data processing and retrieval, k -means is heavily utilized for web-scale semantic deduplication (Abbas et al., 2023) and embedding quantization for late-interaction search (Khattab and Zaharia, 2020; Santhanam et al., 2022b,a). In Large Language Models (LLMs), online clustering is critical for context scaling, enabling dynamic token routing for sparse attention (Roy et al., 2020; Wang et al., 2021; Zhu et al., 2025) and semantic state merging for KV cache compression (Liu et al., 2025; Hooper et al., 2025). More recently, this paradigm has extended to state-of-the-art generative video models, where batched k -means is invoked repeatedly during forward passes to perform semantic-aware token permutation in Diffusion Transformers (Xi et al., 2025; Yang et al., 2025) and extreme low-bit KV-cache quantization for auto-regressive generation (Xi et al., 2026). Consequently, the primary evaluation metric for k -means has gradually shifted from offline analytical throughput to strict online invocation latency.

Algorithmic optimizations for K-Means. To accelerate the clustering process, prior research has focused on reducing the theoretical computational complexity. Methods leveraging the triangle inequality safely skip redundant distance calculations (Elkan, 2003; Ding et al., 2015). Other approaches shrink the effective dataset

size using summarization or sampling to estimate centroid updates (Bachem et al., 2018; Sculley, 2010). Recent theoretical literature also continues to explore dual-distance metrics to improve mathematical convergence (Gada, 2025). While these algorithmic innovations successfully decrease arithmetic operations, standard GPU implementations of k -means remain fundamentally IO-bound, making implementation-level optimization a critical prerequisite to fully translating algorithmic FLOP reductions into end-to-end wall-clock speedups.

Hardware-aware and IO-optimized ML primitives. Recent systems research achieves massive performance breakthroughs by optimizing operator data paths and memory hierarchies rather than altering the underlying mathematical formulas. The most prominent example is FlashAttention (Dao et al., 2022; Dao, 2023; Shah et al., 2024), which fuses attention computation to entirely bypass the explicit materialization of the massive $N \times N$ attention matrix in HBM. This IO-aware philosophy has rapidly expanded to yield heavily optimized primitives for dynamic LLM serving (Ye et al., 2025) and memory management (Kwon et al., 2023). Furthermore, in handling highly dynamic and skewed data distributions, systems frequently transform irregular scatter writes into sorting followed by regular segmented operations to eliminate severe atomic contention (Dao et al., 2023; Guo et al., 2025). These works collectively demonstrate that restructuring an operator’s memory dataflow to respect hardware synchronization and bandwidth limits is the most effective path to accelerating modern AI primitives.

3 Preliminary and Motivation

3.1 Lloyd’s Algorithm and Standard GPU Implementation

Given data points $X \in \mathbb{R}^{N \times d}$ and centroids $C \in \mathbb{R}^{K \times d}$, Euclidean k -means minimizes the objective:

$$\min_{a \in \{1, \dots, K\}^N, C} \sum_{i=1}^N \|x_i - c_{a_i}\|_2^2 \quad (1)$$

Lloyd’s algorithm solves this by alternating between two stages. The first is the **assignment stage**, which computes a distance matrix $D \in \mathbb{R}^{N \times K}$ and assigns each point to the nearest centroid:

$$D_{ik} = \|x_i - c_k\|_2^2, \quad a_i = \arg \min_k D_{ik} \quad (2)$$

The second is the **centroid update stage**, which aggregates the points assigned to each cluster to compute the new centroids:

$$n_k = \sum_{i=1}^N \mathbb{I}[a_i = k], \quad s_k = \sum_{i=1}^N \mathbb{I}[a_i = k] x_i, \quad c_k \leftarrow \frac{s_k}{n_k} \quad (3)$$

In practice, the squared distance in the assignment stage is expanded as $\|x_i - c_k\|_2^2 = \|x_i\|_2^2 + \|c_k\|_2^2 - 2x_i^\top c_k$, which enables the reuse of norms and maps the computation to high-throughput matrix multiplications.

Many existing GPU libraries follow a direct translation of these equations, as outlined in Algorithm 1. It first computes and writes the massive distance matrix D to High Bandwidth Memory (HBM), then reads it back for row-wise reduction. Finally, it performs scatter-style atomic additions at the token granularity to accumulate cluster statistics.

3.2 Kernel-Level Bottlenecks in Standard Implementation

While the standard implementation is mathematically straightforward, our profiling reveals that its end-to-end performance on modern GPUs is severely limited by two low-level bottlenecks: the memory wall in the assignment stage and atomic serialization in the update stage.

Distance materialization bottlenecks the assignment stage. The key inefficiency in the assignment stage is the explicit materialization of the distance matrix $D \in \mathbb{R}^{N \times K}$. The matrix D is a short-lived intermediate that is written to HBM by the distance kernel and immediately read back by the assignment kernel. This dataflow

Algorithm 1: Standard k -means implementation (one iteration)

Input: Data $X \in \mathbb{R}^{N \times d}$, Centroids $C \in \mathbb{R}^{K \times d}$ resident in HBM

Output: Assignments $a \in \{1, \dots, K\}^N$, Updated Centroids C_{new}

// Stage 1: Assignment

Kernel 1 (Distance): Load blocks of X and C from HBM;

Compute $D_{ik} = \|x_i\|_2^2 + \|c_k\|_2^2 - 2x_i^\top c_k$;

Write full matrix $D \in \mathbb{R}^{N \times K}$ to HBM ; // Massive intermediate materialization

Kernel 2 (Argmin): Load row D_{i*} from HBM, Compute $a_i = \arg \min_k D_{ik}$ for each point i ;

Write assignments $a \in \{1, \dots, K\}^N$ to HBM;

// Stage 2: Centroid Update

Kernel 3 (Aggregation): Initialize sum $s \leftarrow 0$, count $n \leftarrow 0$ in HBM;

foreach point index $i \in \{1, \dots, N\}$ *in parallel* **do**

Read feature x_i and assignment a_i from HBM;

 atomic_add(s_{a_i}, x_i); // Severe atomic contention on hot clusters

 atomic_add($n_{a_i}, 1$);

Kernel 4 (Normalization): **Read** s, n from HBM;

Compute $c_k = s_k/n_k$ for each k , and Write C_{new} to HBM;

introduces at least $\Theta(NK)$ writes plus $\Theta(NK)$ reads per iteration, which heavily dominates the total memory traffic when N and K are large.

In contrast, reading the inputs X and C only costs $\Theta(Nd + Kd)$, and writing the outputs costs $\Theta(N + Kd)$. Consequently, even if the distance computation utilizes high-throughput primitives, the end-to-end assignment latency is fundamentally bandwidth-limited by the $2 \cdot \Theta(NK)$ HBM round trips required to materialize D .

Scatter-style updates cause severe atomic write contention. In the centroid update stage, the standard implementation relies on a token-to-cluster aggregation view. Each Cooperative Thread Array (CTA) processes a contiguous block of tokens in their original order, reading the feature x_i and its assigned cluster id a_i , and then issues scatter-style atomic adds to update the global sum and count buffers.

While this implicitly achieves the necessary gather operation, the write path is highly inefficient. Local token blocks often contain repeated and interleaved cluster ids. As a result, multiple warps frequently attempt to update the same centroid simultaneously, leading to irregular and highly concentrated write destinations. This causes severe atomic contention, serialization, and cache-line thrashing. Since atomics are issued at token granularity, the number of atomic operations scales as $O(Nd)$. In our profiling on an NVIDIA H200 GPU, this write-path contention restricts the standard update implementation to only 50 GB/s effective bandwidth, drastically falling short of the hardware’s theoretical reduction bandwidth limits.

3.3 System-Level Constraints in Real Deployments

Beyond the core kernel inefficiencies, k -means faces additional system-level constraints when deployed in modern AI workloads. First, these workloads are frequently invoked with extremely large batch sizes, causing the input data to exceed the GPU’s VRAM capacity. Processing the data in chunks can reduce peak memory usage, but it introduces frequent Host-to-Device (CPU-to-GPU) communication and synchronization, shifting the bottleneck to the PCIe bandwidth. Second, real-world AI pipelines exhibit highly dynamic shapes—the number of points, clusters, and feature dimensions can change rapidly between invocations. Standard implementations that rely heavily on shape specialization require exhaustive auto-tuning to achieve optimal performance. Under dynamic workloads, this configuration search triggers frequent recompilations, significantly increasing the time-to-first-run and hindering practical usability in online scenarios.

4 Methodology

In this section, we introduce flash-kmeans, an efficient k -means implementation for modern GPU workloads. We first present FlashAssign in § 4.1, which fuses distance computation and reduction to eliminate IO overheads. We then present sort-inverse update in § 4.2, which explicitly constructs an inverse mapping to rewrite high-contention atomic scatter into a regular segment-level aggregation. Finally, we discuss several algorithm-system co-design optimizations in § 4.3 to improve deployability in real systems.

4.1 FlashAssign: Materialization-Free Assignment via Online Argmin

To eliminate the severe HBM traffic caused by materializing the distance matrix $D \in \mathbb{R}^{N \times K}$ (§ 3.2), we propose *FlashAssign*. FlashAssign fuses the distance computation and row-wise reduction into a single streaming procedure, ensuring that the full $N \times K$ distance matrix is never explicitly constructed in memory.

Online argmin. FlashAssign relies on an online argmin update. For each point x_i , we maintain two running states in registers: the current minimum distance m_i and the corresponding centroid index a_i . We initialize $m_i = +\infty$ and $a_i = -1$, and then scan the centroids in tiles. For each centroid tile, the kernel computes local distances on chip, identifies the local minimum within that tile, and compares it with the running (m_i, a_i) to keep the smaller one. Repeating this update over all centroid tiles yields the exact global minimum.

Tiling and asynchronous prefetch. FlashAssign uses two-dimensional tiling over both the points and the centroids. For each tile, the kernel maintains the running (m, a) states on chip and scans centroid tiles sequentially. To hide memory latency, we implement double buffer and asynchronous prefetch, ensuring that loading the next centroid tile from HBM overlaps seamlessly with the distance computation of the current tile.

Algorithm 2: FlashAssign (materialization-free assignment)

Input: Data $X \in \mathbb{R}^{N \times d}$, centroids $C \in \mathbb{R}^{K \times d}$, point tile size B_N , centroid tile size B_K

Output: Assignments $a \in \{1, \dots, K\}^N$

Precompute norms $\|x_i\|_2^2$;

foreach point tile X_{tile} of size B_N in parallel **do**

 Initialize on-chip running states: $m \leftarrow +\infty$, $a \leftarrow -1$;

 Prefetch the first centroid tile $C_{\text{tile}}^{(0)}$ from HBM into on-chip buffer;

for $t \leftarrow 0$ to $\lceil K/B_K \rceil - 1$ **do**

if $t + 1 < \lceil K/B_K \rceil$ **then**

 Prefetch $C_{\text{tile}}^{(t+1)}$ into the alternate buffer;

 Compute local distances between X_{tile} and $C_{\text{tile}}^{(t)}$ on chip;

 Compute tile-local minima and indices $(\tilde{m}, \tilde{a})$ for each point in X_{tile} ;

 Update running states using online argmin;

$m \leftarrow \min(m, \tilde{m})$, and update a with the corresponding index;

 Swap buffers;

 Write final assignments a for X_{tile} to HBM;

IO complexity. By fusing distance computation and reduction, FlashAssign fundamentally changes the dataflow. Under ideal streaming execution, each assignment pass only needs to read the point matrix X and centroid matrix C once, and write the assignment vector a once. This reduces the dominant IO complexity from $O(NK)$ to $O(Nd + Kd)$, entirely removing the $2 \cdot \Theta(NK)$ HBM traffic penalty of standard implementations.

4.2 Sort-Inverse Update: Low-Contention Centroid Aggregation

To resolve the severe write-side serialization in the centroid update stage (§ 3.2), we propose *sort-inverse update* as illustrated in Figure 2. Instead of performing scatter-style atomic additions at the token granularity, we explicitly restructure the execution to enable localized, segment-level reductions.

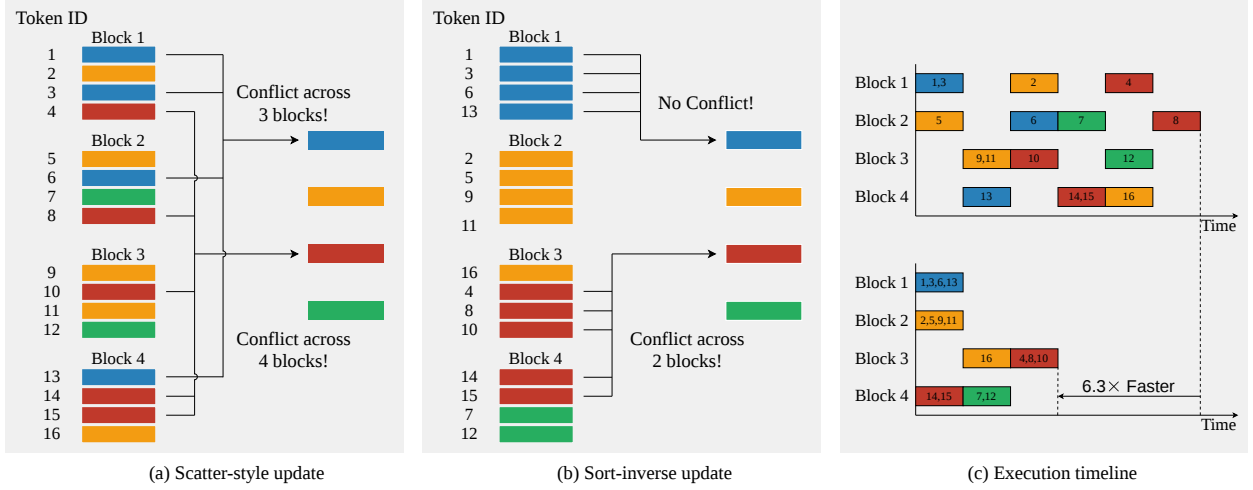


Figure 2 Illustration of the centroid update stage and atomic contention. **(a) Standard scatter-style update:** Tokens are directly scattered to their assigned centroids. The highly irregular mapping leads to severe write-side atomic contention when multiple threads update the same centroid simultaneously. **(b) Sort-inverse update:** flash-kmeans first sorts the tokens by their cluster IDs and constructs an inverse mapping. This transforms the unstructured scatter into regularized, segment-level localized reductions. **(c) Execution timeline comparison:** The timeline reveals that the standard approach stalls frequently due to atomic lock contention on HBM, whereas Sort-Inverse Update issues contention-free memory writes, significantly hiding latency and accelerating the reduction phase.

Explicit inverse mapping via argsort. The core idea is to transform the token-to-cluster update into a cluster-to-token gather. We first apply an `argsort` operation to the assignment vector a to obtain the permutation index `sorted_idx`, and then form the sorted cluster-id sequence a^{sorted} . In this sorted logical order, identical cluster ids naturally group together to form contiguous *cluster-id segments*. Note that this sorting is performed only on the 1D assignment vector a ; we do *not* physically permute the heavy point matrix X in memory.

Segment-level localized aggregation. Once the inverse mapping is built, each CTA processes a contiguous chunk of the sorted sequence a^{sorted} . The CTA identifies the cluster-id segments boundaries inside its assigned chunk and uses `sorted_idx` to gather the corresponding token features from the original X matrix. Crucially, the CTA accumulates partial sums and counts entirely in fast on-chip memory (registers or shared memory) for each segment. It only issues global `atomic_add` operations to HBM at segment boundaries. By keeping the read-side gather from HBM but moving the reduction logic on chip, we effectively bypass the write-path bottleneck. Figure 2(b) provides an overview of this reorganization.

Atomic add count analysis. Sort-inverse update drastically reduces the total number of atomic operations. In a standard scatter update, atomics are issued per token, scaling as $O(Nd)$. In our design, atomics are issued per contiguous cluster-id segment. After sorting, the sequence contains exactly K contiguous segments. Since the workload is partitioned into chunks of size B_N across CTAs, chunk boundaries may split a segment, adding at most $\lceil N/B_N \rceil$ extra boundaries. Therefore, the worst-case number of atomic merges drops to $O((K + \lceil N/B_N \rceil)d)$. This theoretical reduction directly translates to the elimination of write contention.

4.3 Efficient Algorithm-System Co-design

Large-scale data processing via chunked stream overlap. When the input cannot reside entirely in GPU memory and must be staged from CPU memory, we use a chunked stream overlap design. We partition the data into chunks and use CUDA streams to coordinate asynchronous host-to-device transfers and k -means computation. Each chunk is copied to the GPU with non-blocking transfers, then processed by assignment and centroid update kernels, while the next chunk is transferred in parallel, following a double-buffer streaming pattern.

Cache-aware compile heuristic for fast time-to-first-run. Configuration tuning in flash-kmeans compilation often introduces a large time-to-first-run overhead, especially under dynamic shapes and cross-hardware deployment. To address this issue, we design a cache-aware compile heuristic that selects high-quality kernel configurations

Algorithm 3: Sort-Inverse Update (low-atomic centroid update)

Input: Points $X \in \mathbb{R}^{N \times d}$ (original order), assignments $a \in \{1, \dots, K\}^N$, chunk size B_N
Output: Centroid sums $s \in \mathbb{R}^{K \times d}$, counts $n \in \mathbb{R}^K$, updated centroids C
Sort by cluster id: compute `sorted_idx` $\leftarrow$ `argsort`(a);
Construct sorted cluster ids $a^{\text{sorted}}[j] \leftarrow a[\text{sorted_idx}[j]]$;
Initialize $s \leftarrow 0$, $n \leftarrow 0$;
for $l \leftarrow 0$ **to** $N - 1$ *step* B_N **do**
 $r \leftarrow \min(l + B_N, N)$;
 Load $a^{\text{sorted}}[l : r]$ and `sorted_idx`[$l : r$];
 Identify contiguous segments of identical cluster ids in $a^{\text{sorted}}[l : r]$;
 for each segment (u, v, k) **in** $a^{\text{sorted}}[l : r]$ **do**
 Gather token features from original order using indices `sorted_idx`[$u : v$];
 Accumulate local partial sum Δs_k and local partial count Δn_k on chip;
 Atomic merge (once per segment): `atomic_add`($s_k, \Delta s_k$) and `atomic_add`($n_k, \Delta n_k$);
for $k \leftarrow 1$ **to** K **do**
 $c_k \leftarrow s_k / n_k$;

directly from hardware characteristics (in particular, L1 and L2 cache sizes) and problem shape, instead of relying on expensive exhaustive tuning. The goal is not to perfectly tune every shape, but to reliably match near-best configurations at very low compilation cost.

5 Experiments

5.1 Experimental Setup

We evaluate flash-kmeans on an NVIDIA H200 GPU with CUDA 12.8. We benchmark Euclidean k -means over a comprehensive sweep of data sizes N , cluster counts K , feature dimensions D , and batch sizes B . We compare flash-kmeans against four heavily optimized baselines: `fast_pytorch_kmeans`, `fastkmeans` (Clavié and Warner, 2025), NVIDIA cuML (Raschka et al., 2020), and FAISS (Johnson et al., 2019). We report end-to-end latency, individual kernel latencies (assignment and centroid update), and time-to-first-run for dynamic shape deployments.

5.2 Efficiency Evaluation

End-to-end speedup benchmark. Figure 3 compares the end-to-end latency per iteration across implementations under different workloads. For clarity, we group the results into three representative regimes: large N with large K , large N with small K , and small N with small K . In the memory-intensive large- N , large- K regime (e.g., $N = 1\text{M}$, $K = 64\text{K}$, $D = 512$), standard PyTorch implementations run out of memory due to the massive distance matrix, whereas flash-kmeans achieves the largest absolute speedup, outperforming the best baseline (`fastkmeans`) by over $5.4\times$. In the compute-intensive large- N , small- K regime (e.g., $N = 8\text{M}$, $K = 1024$), flash-kmeans remains the fastest, reducing end-to-end latency by **94.4%** (a **17.9** $\times$ speedup over `fast_pytorch_kmeans`). In the small- N , small- K regime, where kernel launch and framework overheads dominate, flash-kmeans still delivers consistent acceleration, scaling up to a **15.3** $\times$ speedup in highly batched scenarios (e.g., $B = 32$). Overall, flash-kmeans remains robust across batch sizes and feature dimensions, demonstrating that its IO-aware optimizations generalize across diverse shape configurations.

Kernel-level efficiency breakdown. To isolate the sources of our end-to-end acceleration, we benchmark the two core k -means stages individually against highly optimized standard implementations. Figure 4 presents the absolute latency comparisons and relative speedups across six representative boundary configurations for each stage. On a highly demanding configuration ($N = 1\text{M}$, $K = 8192$), FlashAssign drastically reduces execution time from 122.5ms down to just 5.8ms, achieving up to a **21.2** $\times$ speedup over the standard assignment implementation. Concurrently, sort-inverse update systematically accelerates the reduction step, reaching up

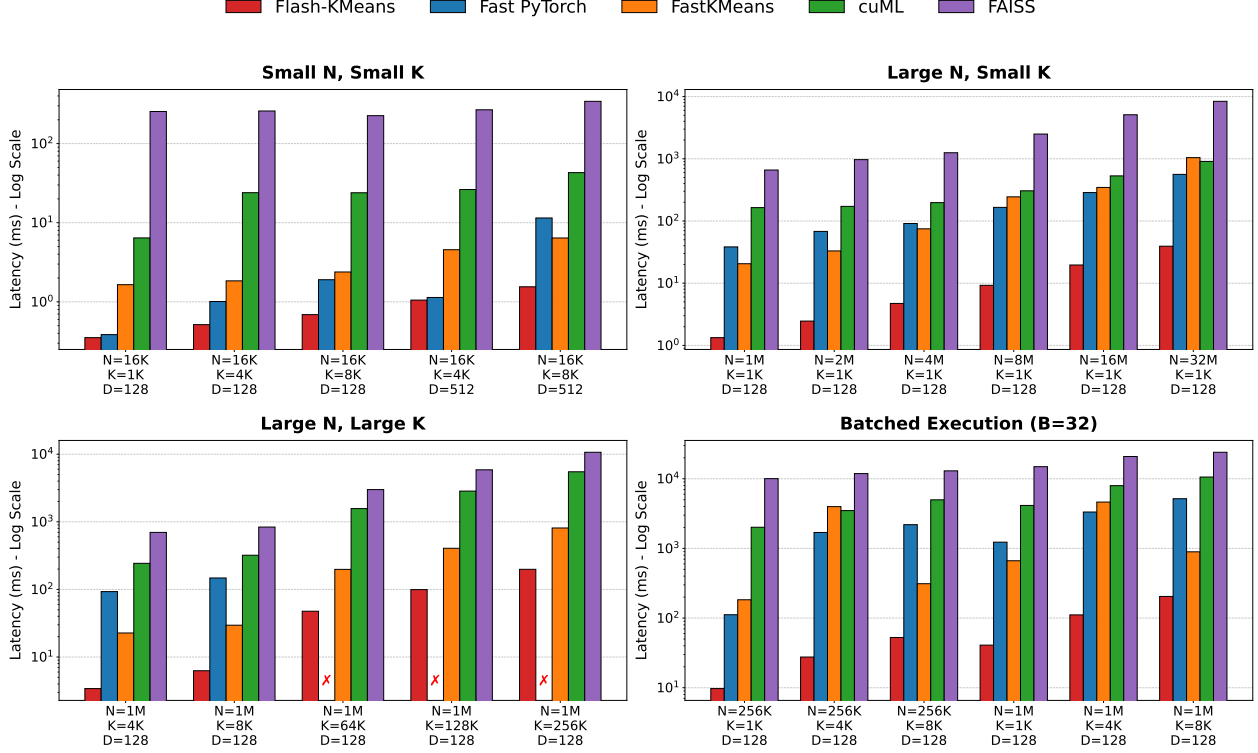


Figure 3 End-to-End Latency of flash-kmeans compared to standard baselines. We group the evaluation into four representative regimes. The y-axis is presented in log scale to accommodate magnitude differences. Red $\times$ marks denote Out-Of-Memory failures (e.g., standard PyTorch fails to explicitly materialize the $N \times K$ distance matrix in Large K workloads). flash-kmeans delivers consistent and substantial speedups across diverse shapes and handles extreme limits gracefully.

to a **6.3 $\times$** speedup on massive-scale workloads (e.g., $B = 1, N = 33M, K = 4096$). These microbenchmarks definitively confirm that flash-kmeans effectively neutralizes both the dominant IO limits and synchronization bottlenecks, aligning directly with our hardware analysis in § 3.2.

5.3 Algorithm-System Co-design Evaluation

Large-scale out-of-core data processing. To evaluate flash-kmeans in memory-constrained, massive-scale scenarios, we benchmark workloads where the dataset severely exceeds GPU VRAM capacity, scaling up to one billion points. Under these extreme conditions, standard PyTorch implementations immediately fail due to Out-Of-Memory errors. We therefore compare flash-kmeans against fastkmeans, the most robust out-of-core baseline available. On an extreme workload of one billion points ($N = 10^9, K = 32768, D = 128$), flash-kmeans completes an iteration in just **41.4 seconds**, whereas the baseline requires 261.8 seconds, yielding a **6.3 $\times$** speedup. Furthermore, on a configuration of $N = 400M$ and $K = 16384$, flash-kmeans achieves an exceptional **10.5 $\times$** end-to-end speedup (8.4s vs. 88.4s). These results demonstrate that our pipelined execution successfully bounds the peak GPU memory footprint while delivering order-of-magnitude acceleration for massive data processing.

Fast time-to-first-run for dynamic shapes. To evaluate our cache-aware compile heuristic, Figure 5 compares its compilation time and runtime performance against exhaustive tuning across six representative scale-up workloads. As shown in the left chart, exhaustive tuning time explodes as the problem size scales, requiring over **325 seconds** to sweep configurations for massive shapes (e.g., $N = 8M, K = 65536$). In contrast, our heuristic analytically derives the optimal configuration in less than **2.5 seconds** across all tested shapes, achieving up to a **175 $\times$** reduction in time-to-first-run. Crucially, as shown in the right chart, this massive reduction in configuration overhead does not compromise execution efficiency. The kernel iteration latency

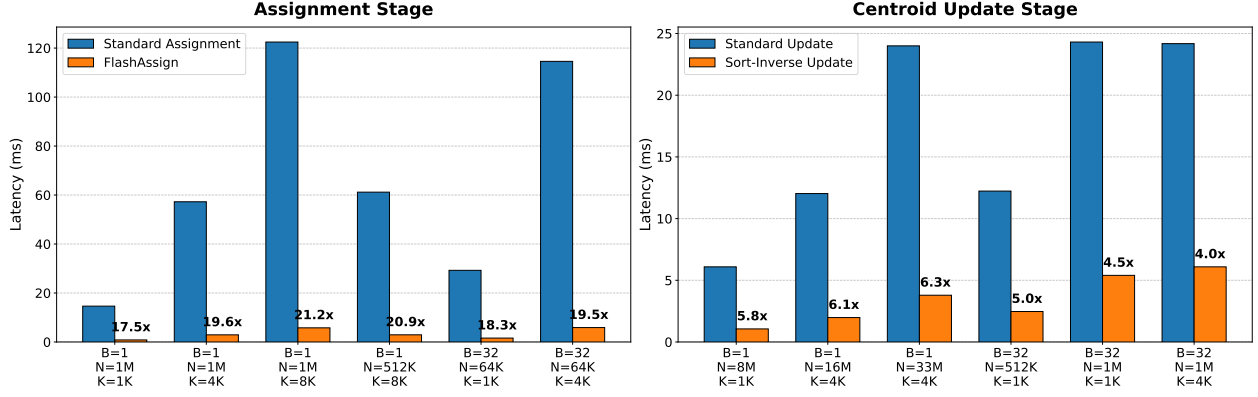


Figure 4 Kernel-level latency breakdown. Latency comparison of our custom kernels versus standard implementations across diverse extreme workloads ($D = 128$). **Left:** FlashAssign completely removes HBM distance materialization, scaling up to a $21.2\times$ speedup. **Right:** Sort-Inverse Update replaces per-token scatter atomic adds with sorted localized merges, eliminating atomic contention and achieving up to a $6.3\times$ speedup.

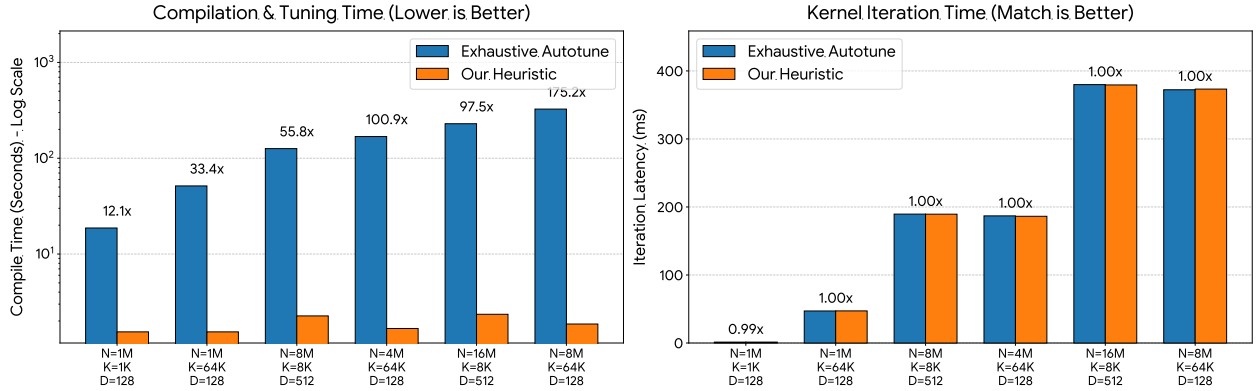


Figure 5 Effectiveness of the cache-aware compile heuristic. **Left:** Compilation and configuration search time (log scale). The heuristic slashes tuning overhead by up to $175\times$, entirely bypassing the severe compilation bottleneck of exhaustive auto-tuning. **Right:** Kernel iteration latency. The heuristic seamlessly matches the optimal performance found by exhaustive search across various shapes, ensuring near-zero runtime degradation.

driven by the heuristic consistently matches the exhaustively tuned oracle, with a negligible performance difference of less than **0.3%**. This confirms that flash-kmeans provides highly deployable, out-of-the-box acceleration for dynamic AI pipelines without requiring offline warm-ups.

6 Conclusion

In this paper, we introduced flash-kmeans, a highly optimized, IO-aware k -means implementation designed to overcome the severe memory and synchronization bottlenecks of modern GPU workloads. Rather than altering the underlying mathematics, we systematically restructured the execution dataflow through FlashAssign to eliminate massive distance matrix materialization, and Sort-Inverse Update to resolve write-side atomic contention. Coupled with an asynchronous out-of-core data pipeline and shape-aware compile heuristics, flash-kmeans delivers up to a $17.9\times$ end-to-end speedup over baselines, and massively outperforms industry standards like cuML and FAISS by $33\times$ and over $200\times$, respectively. By gracefully scaling to extreme workloads of one billion points and slashing compilation overheads by $175\times$ with near-zero performance degradation, flash-kmeans provides a robust, mathematically exact, and highly deployable clustering primitive for next-generation generative AI infrastructure.

References

- Amro Abbas, Kushal Tirumala, Dániel Simig, Surya Ganguli, and Ari S. Morcos. Semdedup: Data-efficient learning at web-scale through semantic deduplication, 2023. <https://arxiv.org/abs/2303.09540>.
- Olivier Bachem, Mario Lucic, and Andreas Krause. Scalable k-means clustering via lightweight coresets, 2018. <https://arxiv.org/abs/1702.08248>.
- Benjamin Clavié and Benjamin Warner. fastkmeans: Accelerated kmeans clustering in pytorch and triton. <https://github.com/AnswerDotAI/fastkmeans/>, 2025.
- Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. <https://arxiv.org/abs/2307.08691>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. <https://arxiv.org/abs/2205.14135>.
- Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference. <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>, Oct 2023. Stanford CRFM Blog.
- Yufei Ding, Yue Zhao, Xipeng Shen, Madanlal Musuvathi, and Todd Mytkowicz. Yinyang k-means: A drop-in replacement of the classic k-means with consistent speedup. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 579–587, Lille, France, 07–09 Jul 2015. PMLR. <https://proceedings.mlr.press/v37/ding15.html>.
- Charles Elkan. Using the triangle inequality to accelerate k-means. In *Proceedings of the Twentieth International Conference on International Conference on Machine Learning*, ICML’03, page 147–153. AAAI Press, 2003. ISBN 1577351894.
- Naitik Gada. A novel k-means clustering approach using two distance measures for gaussian data, 2025. <https://arxiv.org/abs/2511.17823>.
- Wentao Guo, Mayank Mishra, Xinle Cheng, Ion Stoica, and Tri Dao. Sonicmoe: Accelerating moe with io and tile-aware optimizations, 2025. <https://arxiv.org/abs/2512.14080>.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization, 2025. <https://arxiv.org/abs/2401.18079>.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert, 2020. <https://arxiv.org/abs/2004.12832>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. <https://arxiv.org/abs/2309.06180>.
- Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. Clusterkv: Manipulating llm kv cache in semantic space for recallable compression, 2025. <https://arxiv.org/abs/2412.03213>.
- S. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. doi: 10.1109/TIT.1982.1056489.
- J. MacQueen. Some methods for classification and analysis of multivariate observations. In L. M. Le Cam and J. Neyman, editors, *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability - Vol. 1*, pages 281–297. University of California Press, Berkeley, CA, USA, 1967.
- NVIDIA. Nvidia a100 tensor core gpu. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf>, 2021.
- Sebastian Raschka, Joshua Patterson, and Corey Nolet. Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *arXiv preprint arXiv:2002.04803*, 2020.
- Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers, 2020. <https://arxiv.org/abs/2003.05997>.

- Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. Plaid: An efficient engine for late interaction retrieval, 2022a. <https://arxiv.org/abs/2205.09707>.
- Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. Colbertv2: Effective and efficient retrieval via lightweight late interaction, 2022b. <https://arxiv.org/abs/2112.01488>.
- D. Sculley. Web-scale k-means clustering. In *Proceedings of the 19th International Conference on World Wide Web, WWW '10*, page 1177–1178, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781605587998. doi: 10.1145/1772690.1772862. <https://doi.org/10.1145/1772690.1772862>.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. <https://arxiv.org/abs/2407.08608>.
- Shuohang Wang, Luwei Zhou, Zhe Gan, Yen-Chun Chen, Yuwei Fang, Siqi Sun, Yu Cheng, and Jingjing Liu. Cluster-former: Clustering-based sparse transformer for long-range dependency encoding, 2021. <https://arxiv.org/abs/2009.06097>.
- Haocheng Xi, Shuo Yang, Yilong Zhao, Chenfeng Xu, Muyang Li, Xiuyu Li, Yujun Lin, Han Cai, Jintao Zhang, Dacheng Li, Jianfei Chen, Ion Stoica, Kurt Keutzer, and Song Han. Sparse videogen: Accelerating video diffusion transformers with spatial-temporal sparsity, 2025. <https://arxiv.org/abs/2502.01776>.
- Haocheng Xi, Shuo Yang, Yilong Zhao, Muyang Li, Han Cai, Xingyang Li, Yujun Lin, Zhuoyang Zhang, Jintao Zhang, Xiuyu Li, Zhiying Xu, Jun Wu, Chenfeng Xu, Ion Stoica, Song Han, and Kurt Keutzer. Quant videogen: Auto-regressive long video generation via 2-bit kv-cache quantization, 2026. <https://arxiv.org/abs/2602.02958>.
- Shuo Yang, Haocheng Xi, Yilong Zhao, Muyang Li, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Chenfeng Xu, Kelly Peng, Jianfei Chen, Song Han, Kurt Keutzer, and Ion Stoica. Sparse videogen2: Accelerate video generation with sparse attention via semantic-aware permutation, 2025. <https://arxiv.org/abs/2505.18875>.
- Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. Flashinfer: Efficient and customizable attention engine for llm inference serving, 2025. <https://arxiv.org/abs/2501.01005>.
- Kan Zhu, Tian Tang, Qinyu Xu, Yile Gu, Zhichen Zeng, Rohan Kadekodi, Liangyu Zhao, Ang Li, Arvind Krishnamurthy, and Baris Kasikci. Tactic: Adaptive sparse attention with clustering and distribution fitting for long-context llms, 2025. <https://arxiv.org/abs/2502.12216>.